Attitude Estimation and Magnetometer Calibration in the `43-STM32Fxxx_MPU9250_EKF` Module: A Multiplicative Extended Kalman Filter with Ellipsoid Hard/Soft-Iron Calibration

Prepared by Claude (Anthropic) — Source Code Analysis

September 20, 2026

Abstract

This paper presents a detailed mathematical analysis of the navigation and sensor-fusion algorithms implemented in the `43-STM32Fxxx_MPU9250_EKF` firmware module of the `stm32_utilities` repository. The module implements a hardware-agnostic *multiplicative Extended Kalman Filter* (MEKF) for real-time attitude (roll, pitch, yaw) estimation from a 9-axis MPU9250 inertial/magnetic sensor, together with an *ellipsoid least-squares fitting* routine for hard-iron and soft-iron magnetometer calibration, solved through Gauss–Jordan elimination on the associated normal equations. We derive and explain, symbol by symbol, the quaternion kinematics, the linearized error-state dynamics, the discrete-time covariance propagation, the vector-observation measurement update shared by the accelerometer and magnetometer corrections, and the ellipsoid-fitting calibration model. The presentation follows the exact structure of the source code (`ekf.c`, `ekf.h`, `magcal.c`, `magcal.h`, `Gauss_elimination.c`, `imu_port.c`) so that each equation can be traced back to its implementation.

Contents

1 Introduction and System Overview **2**

2 Attitude Representation and Kinematics **3**
- 2.1 Quaternion convention . . . 3
- 2.2 Rotation matrix from quaternion . . . 3
- 2.3 Quaternion kinematics . . . 3

3 The Multiplicative Extended Kalman Filter (MEKF) **4**
- 3.1 State representation: nominal state and error state . . . 4
- 3.2 Filter configuration parameters . . . 4
- 3.3 Time update: state propagation (`ekf_predict`) . . . 5
  - 3.3.1 Bias-corrected rate and quaternion propagation . . . 5
  - 3.3.2 Linearized error-state dynamics . . . 5
  - 3.3.3 Discrete state-transition matrix . . . 5
  - 3.3.4 Covariance time update . . . 5
- 3.4 Measurement update: the generic vector observation . . . 6
  - 3.4.1 Predicted measurement . . . 6
  - 3.4.2 Measurement Jacobian . . . 6
  - 3.4.3 Innovation covariance and Kalman gain . . . 6
  - 3.4.4 Innovation and state correction . . . 6

3.4.5 State injection . . . 7
3.4.6 Covariance update . . . 7
3.5 Accelerometer correction: roll and pitch . . . 7
3.6 Magnetometer correction: heading (yaw) . . . 7
3.7 Euler angle extraction . . . 8
3.8 Algorithm summary . . . 8

4 Magnetometer Hard/Soft-Iron Calibration 8
4.1 Physical motivation . . . 8
4.2 Ellipsoid model . . . 8
4.3 Numerical scaling . . . 9
4.4 Least-squares formulation via normal equations . . . 9
4.5 Solving the normal equations by Gauss–Jordan elimination . . . 9
4.6 Recovering the ellipsoid center (hard-iron offset) . . . 10
4.7 Recovering the per-axis scale (soft-iron correction) . . . 10
4.8 Applying the calibration . . . 10

5 Hardware Interface Layer 10

6 Discussion 11

Symbol Glossary 12

Conclusion 12

1 Introduction and System Overview

The firmware module under analysis implements a classical low-cost *Attitude and Heading Reference System* (AHRS) pipeline for an STM32 microcontroller interfaced with an MPU9250 inertial measurement unit (3-axis gyroscope, 3-axis accelerometer, 3-axis magnetometer). The pipeline consists of two mathematically independent, but functionally connected, algorithms:

(i) A **6-state multiplicative Extended Kalman Filter** (`ekf.c`/`ekf.h`) that fuses angular-rate (gyroscope) integration with vector observations of gravity (accelerometer) and of the local magnetic field (magnetometer) to produce a drift-corrected orientation quaternion and gyroscope bias estimate.

(ii) A **least-squares ellipsoid fit** (`magcal.c`/`magcal.h`), solved via **Gauss–Jordan elimination** (`Gauss_elimination.c`), that removes hard-iron and soft-iron distortion from raw magnetometer readings prior to their use in the EKF.

The hardware-facing layer (`imu_port.c`) merely converts raw sensor register values into physical units (rad/s for the gyroscope, g for the accelerometer, calibrated unit-vector for the magnetometer) and is not part of the estimation mathematics; it is described briefly in Section 5 for completeness.

Remark 1 (Frame conventions)**.** *The filter uses a quaternion* $\boldsymbol{q}$ *that rotates vectors from the* body *frame to the* navigation *frame, in an ENU-like navigation frame in which a level board reads* $+1\,g$ *on the vertical (Z) axis, consistent with the raw output of an MPU9250 lying flat.*

2 Attitude Representation and Kinematics

2.1 Quaternion convention

Orientation is represented by a unit (Hamilton, scalar-first) quaternion

$$\boldsymbol{q} = \begin{bmatrix} w & x & y & z \end{bmatrix}^{\mathsf{T}}, \qquad \|\boldsymbol{q}\| = 1, \tag{1}$$

where w is the scalar part and (x, y, z) the vector part. The quaternion $\boldsymbol{q}$ represents the rotation that maps a vector expressed in the *body* frame into the *navigation* frame.

The Hamilton product of two quaternions $\boldsymbol{a} = [a_0, a_1, a_2, a_3]^{\mathsf{T}}$ and $\boldsymbol{b} = [b_0, b_1, b_2, b_3]^{\mathsf{T}}$, denoted $\boldsymbol{r} = \boldsymbol{a} \otimes \boldsymbol{b}$, is

$$r_0 = a_0 b_0 - a_1 b_1 - a_2 b_2 - a_3 b_3, \tag{2}$$
$$r_1 = a_0 b_1 + a_1 b_0 + a_2 b_3 - a_3 b_2, \tag{3}$$
$$r_2 = a_0 b_2 - a_1 b_3 + a_2 b_0 + a_3 b_1, \tag{4}$$
$$r_3 = a_0 b_3 + a_1 b_2 - a_2 b_1 + a_3 b_0. \tag{5}$$

This is implemented verbatim in `quat_mul()`.

2.2 Rotation matrix from quaternion

The corresponding body-to-navigation rotation matrix $\boldsymbol{R}(\boldsymbol{q}) \in \mathbb{R}^{3\times 3}$ (function `quat_to_R`) is

$$\boldsymbol{R}(\boldsymbol{q}) = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}. \tag{6}$$

A navigation-frame vector $\boldsymbol{v}_{\text{nav}}$ is mapped into the body frame by $\boldsymbol{R}^{\mathsf{T}} \boldsymbol{v}_{\text{nav}}$ (function `R_transpose_mul`), since $\boldsymbol{R}^{\mathsf{T}} = \boldsymbol{R}^{-1}$ for an orthonormal rotation matrix.

2.3 Quaternion kinematics

The continuous-time attitude kinematics are governed by

$$\dot{\boldsymbol{q}} = \tfrac{1}{2}\, \boldsymbol{q} \otimes \begin{bmatrix} 0 \\ \boldsymbol{\omega} \end{bmatrix}, \tag{7}$$

where $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^{\mathsf{T}}$ is the body-frame angular velocity. The filter integrates (7) using a first-order (small-angle) discretization: for a sample interval Δt,

$$\delta\boldsymbol{q} = \begin{bmatrix} 1 \\ \tfrac{1}{2}\omega_x \Delta t \\ \tfrac{1}{2}\omega_y \Delta t \\ \tfrac{1}{2}\omega_z \Delta t \end{bmatrix}, \qquad \boldsymbol{q}_{k+1} = \text{normalize}(\boldsymbol{q}_k \otimes \delta\boldsymbol{q}). \tag{8}$$

This is a first-order truncation of the exact quaternion exponential $\exp\big(\tfrac{1}{2}[0, \boldsymbol{\omega}]\Delta t\big)$, adequate at typical IMU sample rates. Re-normalization after every product compensates for the truncation error and for floating-point drift, keeping $\|\boldsymbol{q}\| = 1$ to numerical precision.

3 The Multiplicative Extended Kalman Filter (MEKF)

3.1 State representation: nominal state and error state

A defining feature of the filter is that the Kalman filter's state vector does *not* contain the quaternion directly. Placing a 4-parameter, norm-constrained quaternion inside an unconstrained linear-Gaussian Kalman state would make the covariance singular and require ad-hoc renormalization of the covariance itself. Instead, following the classical multiplicative EKF formulation (Lefferts–Shuster–Markley), the filter separates:

- a **nominal state**, carried outside the Kalman recursion:

$$\boldsymbol{q} \in \mathbb{R}^4,\ \|\boldsymbol{q}\| = 1, \qquad \hat{\boldsymbol{b}} \in \mathbb{R}^3, \tag{9}$$

  i.e. the current best-estimate quaternion and gyro-bias estimate;

- an **error state**, the actual 6-dimensional Kalman state $\delta\boldsymbol{x} \in \mathbb{R}^6$:

$$\delta\boldsymbol{x} = \begin{bmatrix}\delta\boldsymbol{\theta}\\ \delta\boldsymbol{b}\end{bmatrix}, \qquad \delta\boldsymbol{\theta} = \begin{bmatrix}\delta\theta_x\\ \delta\theta_y\\ \delta\theta_z\end{bmatrix} \in \mathbb{R}^3, \quad \delta\boldsymbol{b} = \begin{bmatrix}\delta b_x\\ \delta b_y\\ \delta b_z\end{bmatrix} \in \mathbb{R}^3. \tag{10}$$

The true attitude is related to the nominal attitude through a small-angle multiplicative correction,

$$\boldsymbol{q}_{\text{true}} \ \approx\ \boldsymbol{q} \otimes \delta\boldsymbol{q}(\delta\boldsymbol{\theta}), \qquad \delta\boldsymbol{q}(\delta\boldsymbol{\theta}) \approx \begin{bmatrix}1\\ \frac{1}{2}\delta\theta_x\\ \frac{1}{2}\delta\theta_y\\ \frac{1}{2}\delta\theta_z\end{bmatrix}, \tag{11}$$

which is exactly the update rule applied after every measurement correction (Section 3.4). This is the mathematical meaning of "multiplicative": the error is *composed* onto the nominal quaternion by quaternion multiplication rather than added component-wise, so unit norm is preserved automatically (up to renormalization for numerical robustness).

The filter's error covariance

$$\boldsymbol{P} \in \mathbb{R}^{6\times 6}, \qquad \boldsymbol{P} = \begin{bmatrix}\boldsymbol{P}_{\theta\theta} & \boldsymbol{P}_{\theta b}\\ \boldsymbol{P}_{b\theta} & \boldsymbol{P}_{bb}\end{bmatrix} \tag{12}$$

is stored row-major as a flat 36-element array (`f->P[36]`) and is initialized to

$$\boldsymbol{P}_0 = \operatorname{diag}\big(\underbrace{\sigma_{\theta,0}^2, \sigma_{\theta,0}^2, \sigma_{\theta,0}^2}_{0.1\ \text{rad}^2},\ \underbrace{\sigma_{b,0}^2, \sigma_{b,0}^2, \sigma_{b,0}^2}_{0.01\ (\text{rad/s})^2}\big) \tag{13}$$

in `ekf_init`, reflecting moderate initial uncertainty in attitude and small initial uncertainty in bias.

3.2 Filter configuration parameters

The structure `ekf_cfg_t` exposes four scalar noise parameters and a reference vector, summarized in Table 1.

Table 1: Filter configuration parameters (`ekf_cfg_t`).

Symbol	Field	Units	Meaning
σ_{gyro}	`gyro_noise`	$\text{rad}\,\text{s}^{-1}/\sqrt{\text{Hz}}$	Gyroscope angle-random-walk (white noise) density
σ_{bias}	`bias_noise`	$\text{rad}\,\text{s}^{-2}/\sqrt{\text{Hz}}$	Gyro bias random-walk (rate random walk) density
σ_{acc}	`accel_noise`	normalized	Isotropic accelerometer measurement-noise variance
σ_{mag}	`mag_noise`	normalized	Isotropic magnetometer measurement-noise variance
$\boldsymbol{r}_{\text{mag}}$	`mag_ref[3]`	unit vector	Local magnetic field direction, navigation frame

3.3 Time update: state propagation (`ekf_predict`)

3.3.1 Bias-corrected rate and quaternion propagation

Given a raw gyroscope sample $\boldsymbol{\omega}_{\text{meas}}$ and time step Δt, the bias-corrected angular rate is

$$\boldsymbol{\omega} = \boldsymbol{\omega}_{\text{meas}} - \hat{\boldsymbol{b}}. \tag{14}$$

The nominal quaternion is propagated using (8) with this bias-corrected $\boldsymbol{\omega}$.

3.3.2 Linearized error-state dynamics

Differentiating the multiplicative error model (11) and the true angular-rate/bias relationship $\hat{\boldsymbol{b}} = \boldsymbol{b}_{\text{true}} - \delta\boldsymbol{b}$ yields the (continuous-time, linearized) error dynamics

$$\dot{\delta\boldsymbol{\theta}} = -\,[\boldsymbol{\omega}\times]\;\delta\boldsymbol{\theta} \;-\; \delta\boldsymbol{b}, \tag{15}$$

$$\dot{\delta\boldsymbol{b}} = \mathbf{0}, \tag{16}$$

where $[\boldsymbol{\omega}\times]$ is the skew-symmetric *cross-product matrix* of $\boldsymbol{\omega} = [\omega_x, \omega_y, \omega_z]^{\mathsf{T}}$,

$$[\boldsymbol{\omega}\times] \;=\; \begin{bmatrix} 0 & -\omega_z & \omega_y \\ \omega_z & 0 & -\omega_x \\ -\omega_y & \omega_x & 0 \end{bmatrix}, \qquad [\boldsymbol{\omega}\times]\;\boldsymbol{v} = \boldsymbol{\omega}\times\boldsymbol{v} \;\;\forall\,\boldsymbol{v}\in\mathbb{R}^3. \tag{17}$$

Equation (15) states that the attitude error rotates with the true body rate and is driven by any uncorrected bias error; (16) models the bias error as constant over the (short) propagation interval, its slow variation being captured instead through injected process noise (below), i.e. a random-walk model.

3.3.3 Discrete state-transition matrix

Writing the linear system $\dot{\delta\boldsymbol{x}} = \boldsymbol{F}\,\delta\boldsymbol{x}$ with

$$\boldsymbol{F} = \begin{bmatrix} -\,[\boldsymbol{\omega}\times] & -\boldsymbol{I}_3 \\ \mathbf{0}_3 & \mathbf{0}_3 \end{bmatrix} \in \mathbb{R}^{6\times 6}, \tag{18}$$

the code uses the first-order discretization $\boldsymbol{\Phi} \approx \boldsymbol{I}_6 + \boldsymbol{F}\,\Delta t$, i.e.

$$\boldsymbol{\Phi} = \left[\begin{array}{c|c} \boldsymbol{I}_3 - [\boldsymbol{\omega}\times]\;\Delta t & -\Delta t\,\boldsymbol{I}_3 \\ \hline \mathbf{0}_3 & \boldsymbol{I}_3 \end{array}\right]. \tag{19}$$

This 6×6 matrix is assembled explicitly in `ekf_predict` (variable `Phi`), with the 3×3 upper-left block containing $\pm\omega_i\Delta t$ terms exactly as in (17), and the upper-right block equal to $-\Delta t\,\boldsymbol{I}_3$.

3.3.4 Covariance time update

The error covariance is propagated by the standard linear Kalman time-update equation

$$\boldsymbol{P}^{-} = \boldsymbol{\Phi}\,\boldsymbol{P}\,\boldsymbol{\Phi}^{\mathsf{T}} + \boldsymbol{Q}, \tag{20}$$

computed in code as two dense 6×6 matrix products (`tmp = Phi * P`, `Pnew = tmp * Phi'`). The discrete process-noise matrix $\boldsymbol{Q}$ is diagonal,

$$\boldsymbol{Q} = \operatorname{diag}\big(\underbrace{q_g, q_g, q_g}_{\text{attitude}},\; \underbrace{q_b, q_b, q_b}_{\text{bias}}\big), \qquad q_g = \sigma^2_{\text{gyro}}\,\Delta t, \qquad q_b = \sigma^2_{\text{bias}}\,\Delta t, \tag{21}$$

which is the usual continuous-white-noise-to-discrete-variance conversion Var $\approx$ PSD $\times\,\Delta t$ for a noise power spectral density integrated over one step.

3.4 Measurement update: the generic vector observation

Both the accelerometer correction (observing gravity) and the magnetometer correction (observing the local magnetic field) share *identical* mathematical structure, implemented once in the function `vector_update`. The general model is: a known, unit-length reference direction $\boldsymbol{r}_{\text{nav}}$ in the navigation frame (gravity $[0, 0, 1]^\mathsf{T}$, or the calibrated magnetic reference $\boldsymbol{r}_{\text{mag}}$) is rotated into the body frame using the *current* attitude estimate, and compared against the actual (normalized) sensor reading.

3.4.1 Predicted measurement

$$\boldsymbol{v}_b = \boldsymbol{R}(\boldsymbol{q})^\mathsf{T}\, \boldsymbol{r}_{\text{nav}}, \tag{22}$$

i.e. the reference vector expressed in the body frame under the current attitude estimate — this is what the sensor *should* read if $\boldsymbol{q}$ were exact.

3.4.2 Measurement Jacobian

Because a small attitude perturbation $\delta\boldsymbol{\theta}$ rotates any navigation-frame-derived body vector according to the first-order approximation

$$\boldsymbol{v}_b(\delta\boldsymbol{\theta}) \approx \boldsymbol{v}_b + \delta\boldsymbol{\theta} \times \boldsymbol{v}_b = \boldsymbol{v}_b + [\boldsymbol{v}_b\times]\ \delta\boldsymbol{\theta}, \tag{23}$$

the observation is linear in the error state with Jacobian

$$\boldsymbol{H} = \begin{bmatrix}[\boldsymbol{v}_b\times] & \mathbf{0}_3\end{bmatrix} \in \mathbb{R}^{3\times 6}. \tag{24}$$

Only the attitude-error block is directly observed; the bias block is corrected indirectly through the cross-covariance $\boldsymbol{P}_{\theta b}$ built up during propagation.

3.4.3 Innovation covariance and Kalman gain

$$\boldsymbol{S} = \boldsymbol{H}\,\boldsymbol{P}\,\boldsymbol{H}^\mathsf{T} + \sigma^2_{\text{noise}}\,\boldsymbol{I}_3 \in \mathbb{R}^{3\times 3}, \tag{25}$$

$$\boldsymbol{K} = \boldsymbol{P}\,\boldsymbol{H}^\mathsf{T}\,\boldsymbol{S}^{-1} \in \mathbb{R}^{6\times 3}, \tag{26}$$

where σ^2_{noise} is `accel_noise` or `mag_noise` depending on the sensor. The 3×3 inverse $\boldsymbol{S}^{-1}$ is computed in closed form via the cofactor/adjugate method (`inv3x3`),

$$\boldsymbol{S}^{-1} = \frac{1}{\det \boldsymbol{S}}\ \text{adj}(\boldsymbol{S}), \tag{27}$$

which is numerically cheap and branch-free for a fixed 3×3 size, in contrast to general Gaussian elimination used elsewhere in the codebase for the larger 6×6 calibration system.

3.4.4 Innovation and state correction

The normalized measurement is $\boldsymbol{z} = \boldsymbol{m}/\|\boldsymbol{m}\|$ (accelerometer or magnetometer reading), and the innovation (residual) is

$$\boldsymbol{y} = \boldsymbol{z} - \boldsymbol{v}_b \in \mathbb{R}^3. \tag{28}$$

The error-state correction is

$$\delta\boldsymbol{x} = \boldsymbol{K}\,\boldsymbol{y} = \begin{bmatrix}\delta\boldsymbol{\theta} \\ \delta\boldsymbol{b}\end{bmatrix} \in \mathbb{R}^6. \tag{29}$$

3.4.5 State injection

The corrected error state is folded back into the nominal state — multiplicatively for attitude, additively for bias:

$$\boldsymbol{q} \leftarrow \text{normalize}\left(\boldsymbol{q} \otimes \begin{bmatrix} 1 & \frac{1}{2}\delta\theta_x & \frac{1}{2}\delta\theta_y & \frac{1}{2}\delta\theta_z \end{bmatrix}^{\mathsf{T}}\right), \tag{30}$$

$$\hat{\boldsymbol{b}} \leftarrow \hat{\boldsymbol{b}} + \delta\boldsymbol{b}. \tag{31}$$

Equation (30) is the concrete instantiation of (11) and is, again, the defining "multiplicative" step of the filter: after every correction the error state is reset to zero (*reset step*, implicit here since $\delta\boldsymbol{x}$ is not itself carried forward) and its information content is transferred entirely into the nominal quaternion.

3.4.6 Covariance update

Since the Kalman gain (26) is computed exactly (not approximately), the simplified (non-Joseph-form) covariance update is used:

$$\boldsymbol{P} \leftarrow \boldsymbol{P} - \boldsymbol{K}\,\boldsymbol{H}\,\boldsymbol{P}. \tag{32}$$

Exploiting the symmetry of $\boldsymbol{P}$, the code observes that $\boldsymbol{HP} = (\boldsymbol{PH}^{\mathsf{T}})^{\mathsf{T}}$, so (32) is evaluated as `P -= K * (P*H`^{T}`)`^{T} without recomputing $\boldsymbol{HP}$ from scratch. After the subtraction, the matrix is explicitly re-symmetrized,

$$\boldsymbol{P} \leftarrow \tfrac{1}{2}\left(\boldsymbol{P} + \boldsymbol{P}^{\mathsf{T}}\right), \tag{33}$$

to counteract the asymmetry that single-precision floating-point round-off accumulates over many iterations — a standard numerical-robustness measure in embedded/real-time Kalman filter implementations.

3.5 Accelerometer correction: roll and pitch

The accelerometer update (`ekf_update_accel`) invokes the generic vector update of Section 3.4 with the navigation-frame reference

$$\boldsymbol{r}_{\text{nav}} = \boldsymbol{g}_{\text{nav}} = \begin{bmatrix} 0 & 0 & 1 \end{bmatrix}^{\mathsf{T}} \tag{34}$$

(gravity direction, unit-normalized). Prior to the update, the raw accelerometer magnitude is gated:

$$\text{reject if} \quad \|\boldsymbol{a}_{\text{meas}}\| < 0.85\,g \quad \text{or} \quad \|\boldsymbol{a}_{\text{meas}}\| > 1.15\,g. \tag{35}$$

This heuristic rejects updates while the vehicle undergoes significant linear acceleration, during which the accelerometer no longer measures gravity alone ($\boldsymbol{a}_{\text{meas}} = \boldsymbol{g} - \boldsymbol{a}_{\text{lin}}$ in the body frame) and would otherwise corrupt the attitude estimate. Because gravity is invariant to rotation about the vertical (Z) axis, this correction observes only roll and pitch, leaving yaw unconstrained by the accelerometer alone.

3.6 Magnetometer correction: heading (yaw)

The magnetometer update (`ekf_update_mag`) uses the same vector-update mechanism with

$$\boldsymbol{r}_{\text{nav}} = \boldsymbol{r}_{\text{mag}} = \texttt{cfg.mag_ref}, \tag{36}$$

the calibrated, unit-normalized local magnetic field direction expressed in the navigation frame (obtained externally, e.g. from a magnetic-declination table or by defining the navigation frame's north axis as the calibrated field direction itself). Because the magnetic field has a horizontal component, this correction is sensitive to heading and thus constrains the yaw degree of freedom left unobserved by gravity, giving the combined filter full 3-axis attitude observability.

3.7 Euler angle extraction

For display or logging, `ekf_euler` recovers Z–Y–X (aerospace) Euler angles from the quaternion:

$$\text{roll } \phi = \operatorname{atan2}\big(2(wx + yz),\, 1 - 2(x^2 + y^2)\big), \tag{37}$$

$$\text{pitch } \theta = \arcsin\Big(\operatorname{clip}\big(2(wy - zx),\, -1,\, 1\big)\Big), \tag{38}$$

$$\text{yaw } \psi = \operatorname{atan2}\big(2(wz + xy),\, 1 - 2(y^2 + z^2)\big), \tag{39}$$

where the argument of arcsin is clamped to $[-1, 1]$ to guard against domain errors from floating-point overshoot near the gimbal-lock singularity at $\theta = \pm 90°$.

3.8 Algorithm summary

Algorithm 2 summarizes one full filter cycle as implemented across `ekf_predict` and `vector_update`.

Table 2: One MEKF cycle (prediction + optional corrections).

Predict (every gyro sample, rate $1/\Delta t$)
1. $\boldsymbol{\omega} \leftarrow \boldsymbol{\omega}_{\text{meas}} - \hat{\boldsymbol{b}}$
2. Integrate $\boldsymbol{q}$ with (8); renormalize
3. Build $\boldsymbol{\Phi}$ via (19)
4. $\boldsymbol{P} \leftarrow \boldsymbol{\Phi}\boldsymbol{P}\boldsymbol{\Phi}^{\mathsf{T}} + \boldsymbol{Q}$ via (20)–(21)
Update (accel) (when $\|\boldsymbol{a}\| \approx 1g$, (35))
1–6. Apply (22)–(32) with $\boldsymbol{r}_{\text{nav}} = [0, 0, 1]^{\mathsf{T}}$, noise σ^2_{acc}
Update (mag) (whenever a calibrated sample is available)
1–6. Apply (22)–(32) with $\boldsymbol{r}_{\text{nav}} = \boldsymbol{r}_{\text{mag}}$, noise σ^2_{mag}

4 Magnetometer Hard/Soft-Iron Calibration

4.1 Physical motivation

An uncalibrated magnetometer, rotated through all orientations at a fixed location, does not trace a sphere centered at the origin. Two distortions are present:

- **Hard-iron effects**: nearby permanently magnetized material adds a constant offset vector to every reading, *shifting* the sphere's center.
- **Soft-iron effects**: nearby ferromagnetic (but not permanently magnetized) material locally distorts the ambient field direction- and orientation-dependently, *stretching/shearing* the sphere into a general ellipsoid.

Both effects, if left uncorrected, make the heading derived from the raw magnetometer vector unusable in the vector-observation update of Section 3.4. The module in `magcal.c` estimates and removes both effects by fitting an ellipsoid to a cloud of raw samples collected while the sensor is rotated through many orientations.

4.2 Ellipsoid model

The calibration assumes an *axis-aligned* ellipsoid model (i.e. no cross-axis/shear terms):

$$x^2 + B\,y^2 + C\,z^2 + P\,x + Q\,y + R\,z + D = 0, \tag{40}$$

where (x, y, z) are raw magnetometer readings (scaled, see below) and $B, C, P, Q, R, D \in \mathbb{R}$ are six unknown parameters to be estimated (the coefficient of x^2 is fixed to 1, which is possible because (40) is defined up to an overall nonzero scale factor).

4.3 Numerical scaling

Raw readings are divided by a constant `MAG_SCALE` = 50 before use:

$$x = \frac{m_x}{\texttt{MAG_SCALE}}, \quad y = \frac{m_y}{\texttt{MAG_SCALE}}, \quad z = \frac{m_z}{\texttt{MAG_SCALE}}. \tag{41}$$

This is a purely numerical conditioning step: fourth-order cross terms such as $x^2 \cdot y^2$ appearing in the normal equations grow very quickly with un-scaled raw counts (readings of order 50 μT give $x^4 \sim 6 \times 10^6$), and summing many such large terms in the accumulators would lose precision even in double arithmetic; rescaling to order unity keeps the normal matrix well conditioned.

4.4 Least-squares formulation via normal equations

Rearranging (40) as a linear system in the unknown parameter vector

$$\boldsymbol{v} = \begin{bmatrix} B & C & P & Q & R & D \end{bmatrix}^{\mathsf{T}} \in \mathbb{R}^6, \tag{42}$$

each sample i contributes one row

$$\boldsymbol{a}_i = \begin{bmatrix} y_i^2 & z_i^2 & x_i & y_i & z_i & 1 \end{bmatrix}, \qquad \text{target}_i = -x_i^2, \tag{43}$$

so that stacking all N samples gives an overdetermined linear system $\boldsymbol{A}\boldsymbol{v} = \boldsymbol{w}$, $\boldsymbol{A} \in \mathbb{R}^{N \times 6}$, solved in the least-squares sense:

$$\boldsymbol{v}^\star = \underset{\boldsymbol{v} \in \mathbb{R}^6}{\arg\min} \|\boldsymbol{A}\boldsymbol{v} - \boldsymbol{w}\|_2^2 \quad \Longleftrightarrow \quad \left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\right) \boldsymbol{v}^\star = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{w}. \tag{44}$$

Rather than storing the (potentially very large) design matrix $\boldsymbol{A}$, the code accumulates the 6×6 normal matrix and 6×1 right-hand side *incrementally*, one sample at a time:

$$\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A} \;\mathrel{+}=\; \boldsymbol{a}_i \boldsymbol{a}_i^{\mathsf{T}}, \qquad \boldsymbol{A}^{\mathsf{T}}\boldsymbol{w} \;\mathrel{+}=\; \boldsymbol{a}_i \cdot \text{target}_i, \tag{45}$$

implemented in `magcal_add` as the fixed-size accumulators `ata[36]` and `atb[6]`. Because (45) has constant memory cost independent of the number of samples N, arbitrarily many calibration samples can be folded in in a streaming fashion, well suited to the limited RAM of an embedded target. The accumulation is performed in `double` precision specifically to avoid catastrophic cancellation when summing many terms; the subsequent solve is carried out in `float`, since by that point the normal matrix is well conditioned.

4.5 Solving the normal equations by Gauss–Jordan elimination

The symmetric 6×6 system $(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A})\boldsymbol{v} = \boldsymbol{A}^{\mathsf{T}}\boldsymbol{w}$ is solved in `magcal_solve` using the routines in `Gauss_elimination.c`: `determinant()` performs elimination with partial pivoting on $\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}$ augmented with the identity matrix $\boldsymbol{I}_6$, and `inversion()` completes the back-substitution, so that the explicit inverse is used to obtain

$$\boldsymbol{v}^\star = \left(\boldsymbol{A}^{\mathsf{T}}\boldsymbol{A}\right)^{-1}\boldsymbol{A}^{\mathsf{T}}\boldsymbol{w} = \begin{bmatrix} B & C & P & Q & R & D \end{bmatrix}^{\mathsf{T}}. \tag{46}$$

A zero (or numerically negligible) determinant indicates a rank-deficient normal matrix — typically from too few or poorly distributed calibration samples — and causes `magcal_solve` to abort.

4.6 Recovering the ellipsoid center (hard-iron offset)

Given the fitted coefficients from (46), the implicit equation (40) is converted to center-offset form by completing the square independently on each axis:

$$(x - x_0)^2 + B(y - y_0)^2 + C(z - z_0)^2 = K, \tag{47}$$

with

$$x_0 = -\frac{P}{2}, \qquad y_0 = -\frac{Q}{2B}, \qquad z_0 = -\frac{R}{2C}, \qquad K = x_0^2 + By_0^2 + Cz_0^2 - D. \tag{48}$$

The point (x_0, y_0, z_0), rescaled back to raw units, is precisely the **hard-iron offset** `c->offset[3]`:

$$\texttt{offset} = \big(x_0,\, y_0,\, z_0\big) \cdot \texttt{MAG_SCALE}. \tag{49}$$

Two validity checks are enforced before accepting the fit:

- $B > \varepsilon$ and $C > \varepsilon$ (with $\varepsilon = 10^{-6}$): a non-positive quadratic coefficient means the fitted quadric is a hyperboloid rather than an ellipsoid, indicating the calibration samples did not sufficiently cover the sphere of orientations;
- $K > \varepsilon'$ (with $\varepsilon' = 10^{-9}$): a non-positive K indicates a degenerate (zero- or imaginary-radius) fit.

4.7 Recovering the per-axis scale (soft-iron correction)

From (47), the semi-axis lengths of the fitted ellipsoid are

$$a_x = \sqrt{K}, \qquad a_y = \sqrt{K/B}, \qquad a_z = \sqrt{K/C}. \tag{50}$$

Dividing each centered coordinate by its own semi-axis length maps the ellipsoid onto the unit sphere, which defines the per-axis **soft-iron scale factors** `c->scale[3]`:

$$\texttt{scale}_x = \frac{1}{a_x \cdot \texttt{MAG_SCALE}}, \qquad \texttt{scale}_y = \frac{1}{a_y \cdot \texttt{MAG_SCALE}}, \qquad \texttt{scale}_z = \frac{1}{a_z \cdot \texttt{MAG_SCALE}}. \tag{51}$$

4.8 Applying the calibration

Given the solved offset and scale, any raw sample is corrected by the affine map (`magcal_apply`)

$$\text{out}_i = \big(\text{raw}_i - \texttt{offset}_i\big) \cdot \texttt{scale}_i, \qquad i \in \{x, y, z\}, \tag{52}$$

which removes the hard-iron shift and normalizes each axis by its soft-iron scale, producing an (approximately) unit-length calibrated field vector directly usable as the measurement $\boldsymbol{m}$ in the magnetometer vector-update of Section 3.4.

5 Hardware Interface Layer

The file `imu_port.c` performs unit conversion only, with no estimation mathematics of its own, and is included here for completeness since it fixes the physical units used throughout Sections 3–4:

- Gyroscope: converted from deg/s (as reported by the vendor driver) to rad/s via multiplication by $\pi/180 \approx 0.0174533$, matching the units expected by `ekf_predict`.
- Accelerometer: left in units of g, matching the $\pm 1\,g$ gating logic of (35) and the unit-gravity reference $\boldsymbol{g}_{\text{nav}} = [0, 0, 1]^{\mathsf{T}}$.

- Magnetometer: read in raw counts scaled by the driver's `MMult` factor (0.1 μT per count), then passed through `magcal_apply` (52) when a valid calibration exists; the absolute physical unit is otherwise immaterial since the vector-update measurement model only uses the *normalized* direction $\boldsymbol{z} = \boldsymbol{m}/\|\boldsymbol{m}\|$.
- Sample interval Δt is measured from consecutive millisecond timestamps and gated to $0 < \Delta t < 0.5\,\mathrm{s}$, rejecting the first sample after initialization and any implausibly large gap (e.g. after a stall).

6 Discussion

The two algorithms analyzed above are textbook instances of well-established techniques, combined into a compact, dependency-free embedded implementation:

- The attitude filter is a standard **6-state indirect (error-state) multiplicative EKF** for quaternion attitude with gyro bias estimation, structurally equivalent to the formulations used in spacecraft attitude determination (Lefferts, Shuster & Markley, 1982) and in many open-source low-cost AHRS/autopilot stacks. Its two measurement updates realize a **TRIAD-style vector-matching correction**: the accelerometer observes the gravity direction (correcting roll/pitch, gated against non-gravitational acceleration), and the magnetometer observes the local field direction (correcting yaw), the two together restoring full 3-axis observability that the gyroscope alone cannot provide due to unbounded integration drift.
- The magnetometer calibration is a standard **linear least-squares ellipsoid fit** solved through the **normal equations**, with the accumulation performed *incrementally* (constant memory) and the fit solved by direct **Gauss–Jordan elimination/matrix inversion**. The resulting affine correction removes both hard-iron (additive) and soft-iron (axis-scaling) distortion, a prerequisite for any downstream heading computation.

A minor point of numerical interest is the deliberate departure from the theoretically more robust *Joseph-form* covariance update in favor of the simpler form (32): this is justified here because the Kalman gain is computed exactly (no approximation or numerical truncation beyond ordinary floating-point rounding), so the simplified form is mathematically equivalent; the subsequent re-symmetrization step (33) is a pragmatic safeguard against the asymmetry that single-precision arithmetic otherwise accumulates over long runtimes on a microcontroller.

Symbol Glossary

Symbol	Source var.	Meaning
$\boldsymbol{q} = [w, x, y, z]^\mathsf{T}$	`f->q[4]`	Unit quaternion, body → navigation frame rotation (Hamilton, scalar-first)
$\boldsymbol{R}(\boldsymbol{q})$	`quat_to_R`	3×3 rotation matrix equivalent of $\boldsymbol{q}$
$\boldsymbol{\omega}$	`w[3]`	Bias-corrected body-frame angular rate (rad/s)
$\boldsymbol{\omega}_{\text{meas}}$	`gyro[3]`	Raw gyroscope measurement (rad/s)
$\hat{\boldsymbol{b}}$	`f->bias[3]`	Estimated gyro bias (rad/s)
$\delta\boldsymbol{\theta}$	`dx[0:3]`	Small-angle attitude error (rad), part of Kalman error state
$\delta\boldsymbol{b}$	`dx[3:6]`	Gyro bias error (rad/s), part of Kalman error state
$\delta\boldsymbol{x}$	`dx[6]`	Full 6-dimensional error state $= [\delta\boldsymbol{\theta}; \delta\boldsymbol{b}]$
$[\boldsymbol{v}\times]$	—	3×3 skew-symmetric cross-product matrix of $\boldsymbol{v}$, s.t. $[\boldsymbol{v}\times]\,\boldsymbol{u} = \boldsymbol{v} \times \boldsymbol{u}$
$\boldsymbol{P}$	`f->P[36]`	6×6 error-state covariance matrix
$\boldsymbol{\Phi}$	`Phi[36]`	Discrete state-transition matrix, $\approx \boldsymbol{I} + \boldsymbol{F}\Delta t$
$\boldsymbol{Q}$	(inline)	Discrete process-noise covariance, diag. from $\sigma_{\text{gyro}}, \sigma_{\text{bias}}$
σ_{gyro}	`cfg.gyro_noise`	Gyro angular-random-walk density (rad/s/$\sqrt{\text{Hz}}$)
σ_{bias}	`cfg.bias_noise`	Gyro bias rate-random-walk density (rad/s^2/$\sqrt{\text{Hz}}$)
σ^2_{acc}	`cfg.accel_noise`	Accelerometer measurement-noise variance (normalized units)
σ^2_{mag}	`cfg.mag_noise`	Magnetometer measurement-noise variance (normalized units)
$\boldsymbol{r}_{\text{mag}}$	`cfg.mag_ref[3]`	Unit local magnetic-field reference direction, nav. frame
$\boldsymbol{g}_{\text{nav}}$	`g_nav[3]`	Unit gravity reference direction, nav. frame $= [0, 0, 1]^\mathsf{T}$
$\boldsymbol{v}_b$	`vb[3]`	Reference vector rotated into the body frame $= \boldsymbol{R}^\mathsf{T}\boldsymbol{r}_{\text{nav}}$ (predicted measurement)
$\boldsymbol{H}$	`H[3x6]`	Measurement Jacobian, $[[\boldsymbol{v}_b\times] \;\; \boldsymbol{0}_3]$
$\boldsymbol{S}$	`S[9]`	Innovation covariance, $\boldsymbol{H}\boldsymbol{P}\boldsymbol{H}^\mathsf{T} + \sigma^2_{\text{noise}}\boldsymbol{I}_3$
$\boldsymbol{K}$	`K[Nx3]`	Kalman gain, $\boldsymbol{P}\boldsymbol{H}^\mathsf{T}\boldsymbol{S}^{-1}$
$\boldsymbol{z}$	`z[3]`	Normalized sensor measurement (unit accel. or unit mag. reading)
$\boldsymbol{y}$	`y[3]`	Innovation/residual, $\boldsymbol{z} - \boldsymbol{v}_b$
B, C, P, Q, R, D	`v[0..5]`	Fitted ellipsoid coefficients (magcal): quadratic (B, C), linear (P, Q, R), constant (D)
x_0, y_0, z_0	`c->offset[3]`	Ellipsoid center = hard-iron offset (raw units)
K (magcal)	(inline `K`)	Constant from completing the square; determines semi-axis lengths
a_x, a_y, a_z	(inline)	Fitted ellipsoid semi-axis lengths
$\texttt{scale}_x, \texttt{scale}_y, \texttt{scale}_z$	`c->scale[3]`	Per-axis soft-iron normalization factors
$\boldsymbol{A}^\mathsf{T}\boldsymbol{A}$, $\boldsymbol{A}^\mathsf{T}\boldsymbol{w}$	`ata[36]`, `atb[6]`	Accumulated normal-equation matrix/vector (magcal)
`MAG_SCALE`	(#define, = 50)	Numerical pre-scaling factor for calibration samples

Table 3: Complete symbol glossary cross-referenced to source-code identifiers.

Conclusion

The `43-STM32Fxxx_MPU9250_EKF` module implements a compact but mathematically complete AHRS stack: a 6-state multiplicative Extended Kalman Filter propagates orientation and gyro bias from angular-rate integration and corrects them with gated, TRIAD-style vector observations of gravity and the local magnetic field; a separate, incrementally-accumulated least-squares ellipsoid fit, solved by Gauss–Jordan elimination, calibrates out hard-iron and soft-iron magnetometer distortion so that its output is suitable as an EKF measurement. Both algorithms are implemented with attention to the numerical realities of embedded single-precision arithmetic — explicit renormalization of the quaternion, re-symmetrization of the covariance, double-precision accumulation of the calibration normal equations, and closed-form small-matrix inversions in place of general-purpose solvers where the matrix size permits it.